

```

<kedsI_v6_document>
  <metadata>
    <target_hardware>NVIDIA RTX 5070 12GB (Single Board)</target_hardware>
    <compute_capability>9.x (Ada/Blackwell Architecture)</compute_capability>
    <software_stack>CUDA Toolkit 13.3, CUDA Python 1.0, cuTile</software_stack>
    <core_objective>Build `nano-vllm` multi-user continuous batching inference engine.</core_o
  </metadata>

  <knowledge_graph>
    <concept id="Green_Contexts" desc="CUDA 13.3 feature. Physically partitions Streaming Mult
    <concept id="CUDA_Graph" desc="Records/replays GPU execution to eliminate Python API launc
    <concept id="Interference" desc="In multi-tenant LLM, User A's heavy Prefill ( $O(N^2)$ ) bloc
    <relation source="Green_Contexts" target="Interference" type="resolves" detail="Isolates P
  </knowledge_graph>

  <milestones>
    <stage name="1_FMHA" complexity="Low">
      <bottleneck>SM Starvation on small sequence length.</bottleneck>
      <solution>Implement `cutile` with Split-K or Flash-Decoding style Sequence-dimension par
    </stage>

    <stage name="2_Single_User_LLM" complexity="Medium">
      <bottleneck>High Python Driver API overhead during token-by-token Decoding loop.</bottle
      <solution>
        <code_primitive lang="python">
          # CUDA Graph Capture for Decode Phase
          # Ensure static memory allocation before capture!
          g = torch.cuda.CUDAGraph()
          # Warmup
          for _ in range(3): model(static_input, static_kv_cache)
          # Capture
          with torch.cuda.graph(g):
              static_logits = model(static_input, static_kv_cache)

          # Generation Loop
          for _ in range(max_len):
              static_input.copy_(next_token)
              g.replay()
              next_token = sample(static_logits)
        </code_primitive>
      </solution>
    </stage>

    <stage name="3_Nano_VLLM_Multi_Tenant" complexity="Extreme">
      <bottleneck_1>VRAM Limit: 12GB on RTX 5070 causes instant OOM with standard KV Cache.</b
      <bottleneck_2>Compute Interference: Continuous batching mixes heavy Prefills and light D

      <knowhow_1 id="Paged_cuTile">
        Change `ct.load` in cuTile from contiguous memory to Indirect Addressing using a `Bloc
      </knowhow_1>

      <knowhow_2 id="Green_Context_Isolation">
        <description>
          Use CUDA Python 1.0 to partition the RTX 5070's SMs. Assign ~80% to a 'Prefill Conte

```

```
</description>
<code_primitive lang="python">
    from cuda.core import ContextOptions, SMResourceOptions
    import cuda.device as dev

    sm = dev.resources.sm
    total_sms = sm.sm_count
    decode_sms = 16 # Critical group for latency
    prefill_sms = total_sms - decode_sms

    # Split SMs into disjoint groups
    split_result = sm.split(SMResourceOptions(count=(prefill_sms, decode_sms)))
    prefill_grp, decode_grp = split_result[0]

    # Create isolated contexts
    ctx_prefill = dev.create_context(ContextOptions(resources=[prefill_grp]))
    ctx_decode = dev.create_context(ContextOptions(resources=[decode_grp]))

    # nano-vllm Scheduler Logic:
    # if request.is_prefill():
    #     with ctx_prefill: run_cutile_fmha()
    # if request.is_decode():
    #     with ctx_decode: run_cuda_graph_replay()
</code_primitive>
</knowhow_2>
</stage>
</milestones>

<agent_eval_loop>
    <rule id="1">Start the server. Send 1 huge prompt (Prefill, 4000 tokens) and 1 ongoing gen
    <rule id="2">Measure Inter-Token Latency (ITL) of the Decode task.</rule>
    <rule id="3">SUCCESS_CRITERIA: If ITL spikes > 50ms during the Prefill, Green Context p
    <rule id="4">VRAM_CRITERIA: If Peak VRAM > 11.5GB, reduce Paged KV Cache block size and
</agent_eval_loop>
</keds_l_v6_document>
```

1. 두 DSL의 구조 및 철학 비교

항목	v5-DSL (YAML 기반 시스템 모델)	v6-DSL (XML 기반 마일스톤 지식)
주요 목적	자동 튜닝 및 아키텍처 생성 파이프라인 (Design Space → Tuning → Build)	단계별 문제 해결 가이드 및 구현 예시 (Milestone → Solution → Code)
표현 방식	추상적인 <code>design_space</code> , <code>tuning_space</code> , <code>knowledge</code> , <code>agent_loop</code>	구체적인 <code><stage></code> , <code><bottleneck></code> , <code><solution></code> , <code><code_primitive></code>
코드 포함	없음 (파라미터만 열거)	실제 Python 코드 스니펫 다수 포함 (CUDA Graph, Green Contexts 등)
규칙 표현	<code>knowledge.fact</code> 에 논리적 assertion 및 action	<code><agent_eval_loop></code> 에 평가 규칙 및 성공 기준
확장성/모듈화	구조화된 블록으로 모듈화 가능 (design, tuning, build)	마일스톤 기반으로 순차적 복잡도 증가

항목	v5-DSL (YAML 기반 시스템 모델)	v6-DSL (XML 기반 마일스톤 지식)
자동화 연계	<code>engineering_loop</code> 를 통한 그리드 서치, 메트릭 측정 명시적	평가 루프가 있지만 튜닝 파라미터 공간이 구체적이지 않음
대상 독자	DSL 파서 + 오토튜너 (기계 해석 우선)	코딩 에이전트 (LLM) + 사람 개발자 (이해와 구현 중심)

2. 코딩 에이전트 관점에서의 실효성 비교

2.1 v5-DSL의 강점과 약점

- 강점:** 매우 체계적. 에이전트가 이 DSL을 해석할 수 있다면, 자동으로 파라미터 조합을 생성하고 벤치마크하며 최적의 커널을 빌드할 수 있습니다. **자율적 최적화 파이프라인**에 적합.
- 약점:** 에이전트가 이 DSL을 이해하려면 추가적인 **프롬프트 엔지니어링**이 필요. 자연어가 아니므로 LLM이 의미를 정확히 해석하지 못할 가능성이 큼. `knowledge` 의 `assertion-action`을 코드로 번역하는 것은 매우 높은 수준의 추론 능력을 요구함. 실제로 코딩 에이전트에게 주입하기에는 추상화가 과도함.

2.2 v6-DSL의 강점과 약점

- 강점:**
 - 코드가 곧 지식:** `<code_primitive>`에 담긴 실제 코드는 에이전트가 곧바로 모방·참조할 수 있습니다.
 - 단계적 접근:** 복잡도가 낮은 Stage 1부터 순차적으로 해결하는 구조이므로, 에이전트가 이전 단계 성공을 기반으로 점진적으로 학습·구현할 수 있습니다.
 - 명확한 성공 기준:** `<agent_eval_loop>`의 ITL, VRAM 기준은 에이전트가 자기 평가를 통해 디버깅하도록 유도함.
 - LLM 친화적:** XML 태그는 LLM이 정보를 구조적으로 파싱하는 데 도움을 주며, 자연어 설명과 코드가 혼합되어 있어 **In-Context Learning**에 이상적입니다.
- 약점:**
 - 자동 튜닝 파이프라인 부재:** 파라미터 공간이 구체적이지 않아, 에이전트가 수동으로 튜닝할 때 지침이 부족할 수 있습니다.
 - 확장성:** 여러 종류의 최적화를 추가하려면 XML 구조가 복잡해질 수 있습니다.
 - 특정 버전 의존:** CUDA 13.3, cuTile, RTX 5070 등에 강하게 결합되어 있어, 다른 환경에서는 재사용이 어려울 수 있습니다.

3. Full ICL 주입을 위한 더 나은 대안 분석

코딩 에이전트에게 커널 엔지니어링 지식을 가장 효과적으로 전달하기 위해서는 다음과 같은 원칙을 충족해야 합니다.

- 실행 가능한 예제 (Executable Examples)**
에이전트는 설명보다 코드를 더 잘 모방합니다.
→ v6-DSL의 `<code_primitive>`는 이 점에서 매우 우수함.
- 계층적 지식 구조 (Hierarchical Knowledge)**
복잡한 최적화는 작은 성공의 연속임.
→ v6-DSL의 `<milestones>` (1_FMHA → 2_Single_User → 3_Multi_Tenant)는 에이전트가 **커리큘럼** 방식으로 학습하도록 유도할 수 있음.
- 자기 평가 및 피드백 루프 (Self-Evaluation)**
에이전트가 생성한 코드를 스스로 검증할 수 있어야 함.
→ v6-DSL의 `<agent_eval_loop>`는 훌륭하지만, v5-DSL의 `engineering_loop`처럼 파라미터 탐색 공간과 결합되면 더 강력해짐.
-

제약 조건의 명시적 선언 (Explicit Constraints)

레지스터 개수, 메모리 대역폭, SM 개수 등 하드웨어 제약을 에이전트가 절대 위반하지 않도록 해야 합니다.

→ v5-DSL의 `constraint` 블록이 이 역할을 하지만, v6-DSL에는 명시적 수치 제약이 부족합니다. 이를 `agent_eval_loop`에 포함시키면 좋습니다.

가장 유망한 대안: “하이브리드 구조화 프롬프트” (Hybrid Structured Prompt)

v6-DSL의 실용성과 v5-DSL의 체계성을 결합하여, 아래와 같은 계층적 ICL 템플릿을 코딩 에이전트의 시스템 프롬프트로 주입하는 것입니다.

```
<kernel_engineering_guide>
  <hardware_constraints>
    GPU: RTX 5070 12GB, SM count: 48, Max registers: 255, Max shared mem: 100KB
    Critical: tile_m=128 causes register spill. Prefer 64.
  </hardware_constraints>

  <task_milestones>
    <milestone id="1_fmha">
      <goal>Implement FlashAttention with cuTile, SM starvation issue.</goal>
      <allowed_params>tile_m=64, tile_n=64 or 128, split_k=1-4</allowed_params>
      <code_ref>AttentionFMHA.py from cuTile samples</code_ref>
    </milestone>
    <milestone id="2_single_user_llm">
      <goal>Integrate into nano-vllm, add CUDA Graph for decode loop.</goal>
      <code_snippet>... (CUDA Graph capture code) ...</code_snippet>
    </milestone>
    <milestone id="3_nano_vllm_multi_tenant">
      <goal>Continuous batching + PagedAttention + Green Contexts.</goal>
      <code_snippet>... (Paged_cuTile and Green Contexts API code) ...</code_snippet>
      <eval_criteria>
        Decode ITL < 50ms even during Prefill of 4000 tokens.
        Peak VRAM < 11.5GB.
      </eval_criteria>
    </milestone>
  </task_milestones>

  <tuning_protocol>
    To find optimal performance, iterate over:
    prefill_tile_m = [64], tile_n = [64,128]
    decode_kv_block = [16,32], split_cta = [1,2,4]
    Measure tokens/sec using bench_nano_vllm.py.
    Stop when register count <= 255 and latency target met.
  </tuning_protocol>
</kernel_engineering_guide>
```

이 템플릿은 에이전트가 **순서대로 작업을 진행하면서**, 각 단계에서 필요한 코드와 제약을 **명확히 참조**할 수 있습니다. 또한 필요하다면 `<tuning_protocol>`을 기반으로 자동화 스크립트를 **에이전트가 직접 생성**하도록 유도할 수 있습니다.

4. 결론: 어떤 DSL이 코딩 에이전트에 더 적합한가?

- ****v5-DSL (YAML 기반)****은 자동화 시스템 구축에는 이상적이지만, 현재의 LLM 코딩 에이전트가 이를 해석하여 자율적으로 커널을 설계하기에는 **추상화 수준이 너무 높고 비직관적**입니다.
- ****v6-DSL (XML 기반 마일스톤)****은 코드 예제와 단계별 목표를 담고 있어 에이전트가 **즉시 생산적인 코드를 작성**하도록 이끌기에 훨씬 효과적입니다. 따라서, 코딩 에이전트에게 “**Full ICL**”로 커널 엔지니어링 노하우를 주입하려면, v6-DSL을 기본 틀로 삼고, v5-DSL의 **명시적 제약 조건**과 튜닝 탐색 공간을 추가한 **하이브리드 구조화 프롬프트**입니다.